

CODEALPHA CYBERSECURITY INTERNSHIP

TASK 1 — BASIC NETWORK SNIFFER

PROJECT REPORT

Name: Abdullahi Salisu Lawan

Internship: CodeAlpha Cybersecurity Internship

Task: Task 1 — Basic Network Sniffer

Environment: Kali Linux Virtual Machine

Primary Technology: Python / Scapy

2. Introduction

This project was completed as Task 1 of the CodeAlpha Cybersecurity Internship: Basic Network Sniffer. The task focuses on building a Python program capable of capturing network traffic and analyzing packet information. The project provides practical exposure to how data moves through a network and how common network protocols can be observed at packet level.

The implementation was carried out in a Kali Linux virtual-machine laboratory environment using Python and the Scapy packet-manipulation library. Testing activities included ICMP ping traffic, DNS resolution using nslookup, and observation of a web request through browser Developer Tools.

3. Project Objective

The main objective was to develop and test a basic network sniffer that can observe network packets and present useful information about them. In particular, the project focused on identifying source and destination IP addresses, recognizing protocols, displaying packet length, and handling available payload information.

The project also aimed to develop practical understanding of network traffic, packet structure, and basic protocol behavior in a controlled lab environment.

4. Tools and Technologies

- Kali Linux — cybersecurity-focused Linux operating system used as the laboratory environment.
- Python 3 — programming language used to implement the network sniffer.
- Scapy — Python library used for packet capture and packet inspection.
- Linux Terminal — used to run commands, tests, and the sniffer.
- ping — used to generate ICMP network traffic.
- nslookup — used to generate DNS-related traffic and verify name resolution.
- Browser Developer Tools — used as supplemental evidence to observe an HTTP POST request and its request data.

5. Lab Environment

The project was performed inside a Kali Linux virtual machine. The captured-packet evidence shows the Kali VM using the private address 10.0.0.5 during the observed traffic. The laboratory setup allowed network traffic generated by the VM to be monitored without requiring access to third-party systems.

The work was treated as a controlled educational exercise. Only traffic generated or observed within the authorized lab environment was used for the project evidence.

6. Installation

Python 3 was verified from the Kali Linux terminal, with the supplied screenshot showing Python 3.14.7. Scapy was used by the completed sniffer implementation for packet capture and inspection. The successful packet-capture output confirms that the required runtime and packet-capture environment were operational.

The installation workflow consisted of verifying Python, updating Kali package information where required, installing the Python Scapy package, and verifying the library before running the sniffer.

7. Network Interface Configuration

The network interface used by the Kali VM was identified before packet capture so that traffic could be monitored. The captured results show the VM participating in network communication using the private IP address 10.0.0.5.

The supplied evidence does not contain a separate screenshot of the `ip addr` interface-identification command, so no additional interface screenshot is claimed in this report.

8. Sniffer Implementation

The sniffer was implemented in Python using Scapy. The program captures packets and processes packets containing an IP layer. For each relevant packet, it extracts the source IP address, destination IP address, protocol information, and total packet length. When a raw payload is present, the program also attempts to display a readable representation of part of the payload.

The program was executed with the required privileges so that it could access network traffic from the Kali VM.

9. Code Explanation

The implementation follows a simple packet-processing workflow:

1. Import Scapy packet-capture functionality.
2. Define a packet callback function.
3. Check whether the packet contains an IP layer.
4. Extract source and destination IP addresses.
5. Identify TCP, UDP, and ICMP traffic where applicable.
6. Calculate packet length.
7. Display available raw payload information in a limited form.
8. Start packet capture and pass each packet to the callback.

10. Packet Capture

After the sniffer was started, network traffic was generated from the Kali VM. The captured-packet evidence shows traffic involving 10.0.0.5 and external destinations. TCP packets were observed with packet lengths such as 54, 60, and 83 bytes in the supplied output.

The sniffer also displayed payload information when raw data was available. This demonstrates that packet capture can expose more than addressing information, which is why packet captures must be handled carefully.

11. Packet Analysis

The captured packets were analyzed using four main fields:

Source IP — identifies the IP address from which the packet originated.

Destination IP — identifies the IP address toward which the packet was sent.

Protocol — identifies the protocol observed by the sniffer, including TCP in the supplied packet evidence and ICMP during the ping test.

Packet Length — shows the total size of the captured packet as processed by the program.

The evidence also demonstrates that packet payload data may sometimes be available. Payload content can be binary or otherwise unreadable, and modern encrypted communications should not be assumed to expose their application data in readable form.

12. Testing

Two command-line tests were performed. First, an ICMP ping test was performed against 8.8.8.8 using four echo requests. One supplied run shows four packets transmitted, one packet received, and 75% packet loss. A later supplied run shows four packets transmitted and three packets received, resulting in 25% packet loss.

Second, nslookup was used against example.com. The output returned DNS information for example.com, including IPv4 and IPv6 addresses. This provided a second type of network activity for observation.

The packet-capture evidence also shows TCP traffic generated while the Kali system communicated with external destinations.

13. Results

The task successfully demonstrated packet-level network observation in the Kali Linux lab.

The results included:

- Python 3.14.7 verified in Kali Linux.
- ICMP traffic generated using ping.
- DNS resolution activity for example.com using nslookup.
- Captured TCP traffic involving the Kali VM address 10.0.0.5.
- Display of source IP, destination IP, protocol, packet length, and available payload information.
- Supplemental observation of a browser POST request through Developer Tools.

14. Screenshots / Evidence

The supplied evidence supports the major stages of the task: Python verification, ICMP testing, captured packet output, DNS testing, and supplemental browser-request observation. The browser evidence has been redacted before publication because the original screenshot visibly contained test credential values.

Figure 1 — Python 3.14.7 environment

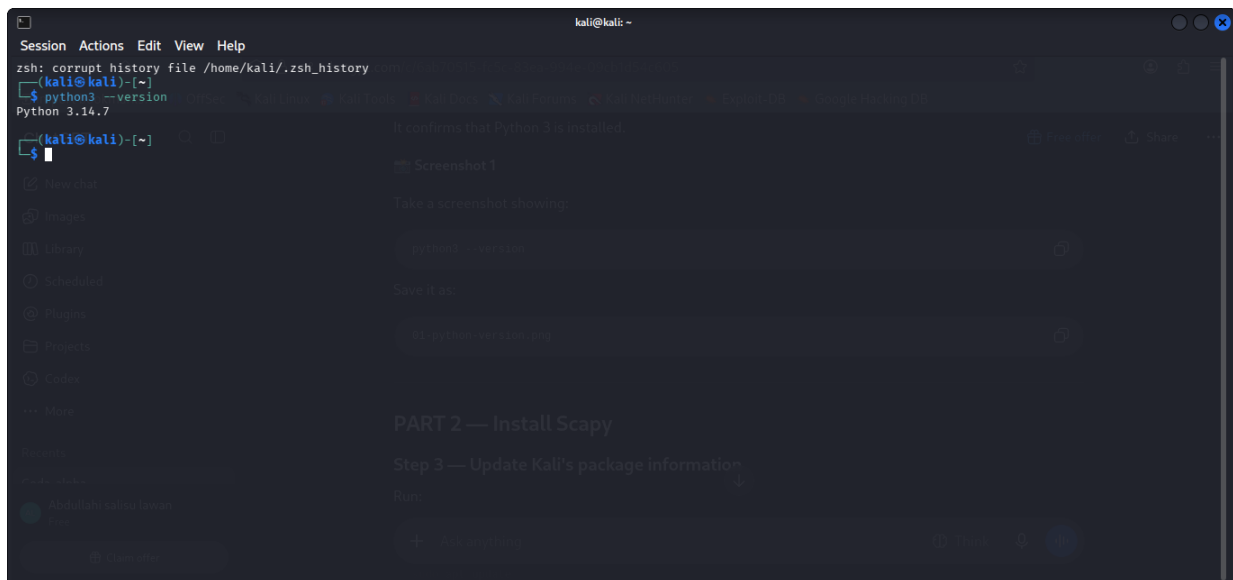


Figure 2 — ICMP ping test

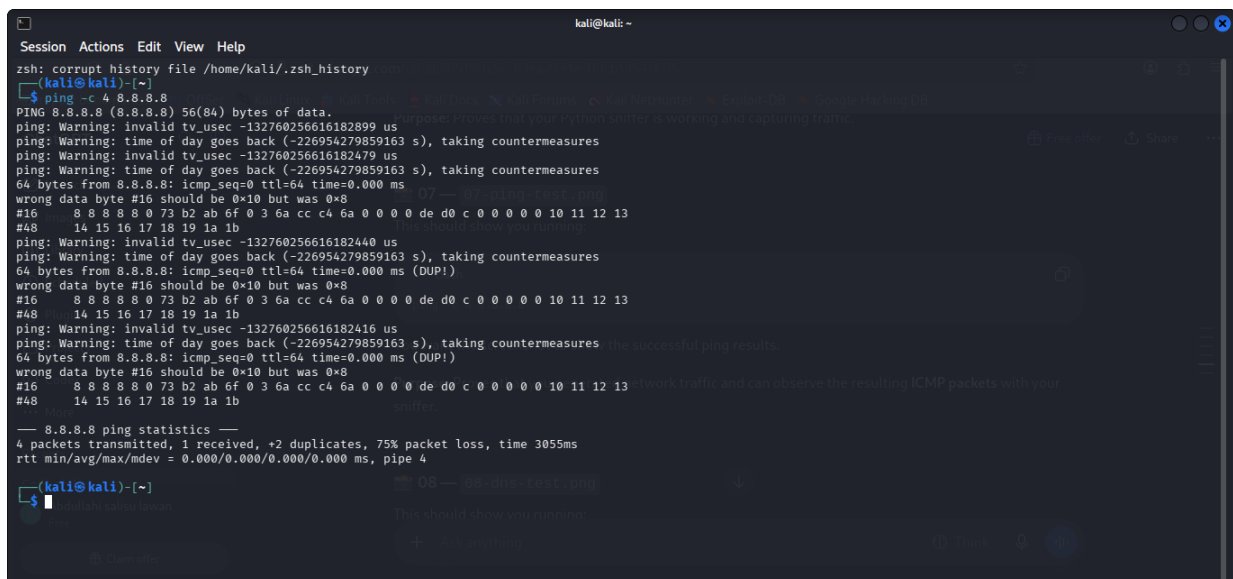


Figure 3 — Captured network packets

```
kali@kali: ~  
Session Actions Edit View Help  
-----  
Source IP      : 10.0.0.5  
Destination IP : 146.75.89.91  
Protocol       : TCP  
Packet Length  : 54  
-----  
Source IP      : 10.0.0.5  
Destination IP : 146.75.89.91  
Protocol       : TCP  
Packet Length  : 54  
-----  
Source IP      : 146.75.89.91  
Destination IP : 10.0.0.5  
Protocol       : TCP  
Packet Length  : 60  
-----  
Source IP      : 10.0.0.5  
Destination IP : 140.82.112.26  
Protocol       : TCP  
Packet Length  : 83  
Payload        : *SDej*****;bC_W  
-----  
Source IP      : 140.82.112.26  
Destination IP : 10.0.0.5  
Protocol       : TCP  
Packet Length  : 60  
-----  
(kali@kali)~$
```

Figure 4 — DNS resolution test

```
kali@kali: ~  
Session Actions Edit View Help  
-----  
zsh: corrupt history file /home/kali/.zsh_history  
(kali@kali)~$ nslookup example.com  
Server:      192.168.215.251  
Address:     192.168.215.251#53  
Non-authoritative answer:  
Name:   example.com  
Address: 172.66.147.243  
Name:   example.com  
Address: 104.20.23.154  
Name:   example.com  
Address: 2606:4700:10::ac42:93f3  
Name:   example.com  
Address: 2606:4700:10::6814:179a  
(kali@kali)~$
```

Figure 5 — Browser login POST request (credentials redacted)

about:sessionrestore x Patient Portal Login | Mediro: x

medirozahospital.com/patient/login.php

Import bookmarks... OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

Username not found

Username

Password

Show password

Inspector Console Debugger Network Style Editor Performance Memory Storage Accessibility Application

Status	Method	Domain	File	Initiator	Type	Size
200	POST	medirozahospital.com	login.php	document	html	2.28 kB
200	GET	medirozahospital.com	styles.css	stylesheet	css	1.55 kB
404	GET	medirozahospital.com	favicon.ico	FaviconLoader	html	1.25 kB

Headers Cookies Request Response Timings Security

Filter Request Parameters

Form data

username: "Test@example.com"
password: "Test123!"

Form data [redacted for publication]

3 requests 5.08 kB / 1.22 kB transferred Finish: 3.11 s DOMContentLoaded: 2.80 s load: 2.84 s

15. Challenges Encountered

One challenge during testing was inconsistent network response during the ICMP ping test. The supplied evidence shows packet loss, with one run showing 75% loss and another showing 25% loss. This demonstrates that network response can vary during testing.

Another practical challenge was interpreting packet information because payload data can contain binary or encoded values rather than immediately readable text.

16. Troubleshooting

Troubleshooting focused on verifying the environment before interpreting packet results. Python was checked first. Network activity was then generated using known commands such as ping and nslookup so that the sniffer could be tested against recognizable traffic.

When packet output appeared, source and destination addresses, protocol labels, packet length, and payload display were reviewed to confirm that the sniffer was processing packets. The browser Developer Tools screenshot was treated as supplemental network observation rather than proof of a vulnerability.

17. Security Lessons Learned

This project provided several important cybersecurity lessons:

- Network traffic can be observed at packet level.
- Source and destination addressing provides useful context about communication.
- Different protocols produce different types of traffic.
- Packet payloads may contain sensitive information.
- Network sniffing should only be performed on systems and networks where monitoring is authorized.
- Credentials and other sensitive data should never be published in GitHub evidence.
- Packet capture is useful for security monitoring, troubleshooting, and understanding network behavior, but captured information must be handled responsibly.

18. Conclusion

The CodeAlpha Basic Network Sniffer task was completed as a controlled cybersecurity laboratory exercise. A Python and Scapy-based approach was used to observe network traffic and extract useful packet information. Testing with ICMP ping and DNS resolution provided recognizable traffic patterns, while the captured-packet evidence demonstrated source and destination IP addresses, protocol information, packet lengths, and available payload data.

The project strengthened practical understanding of packet capture, network protocols, traffic analysis, and responsible handling of network evidence.